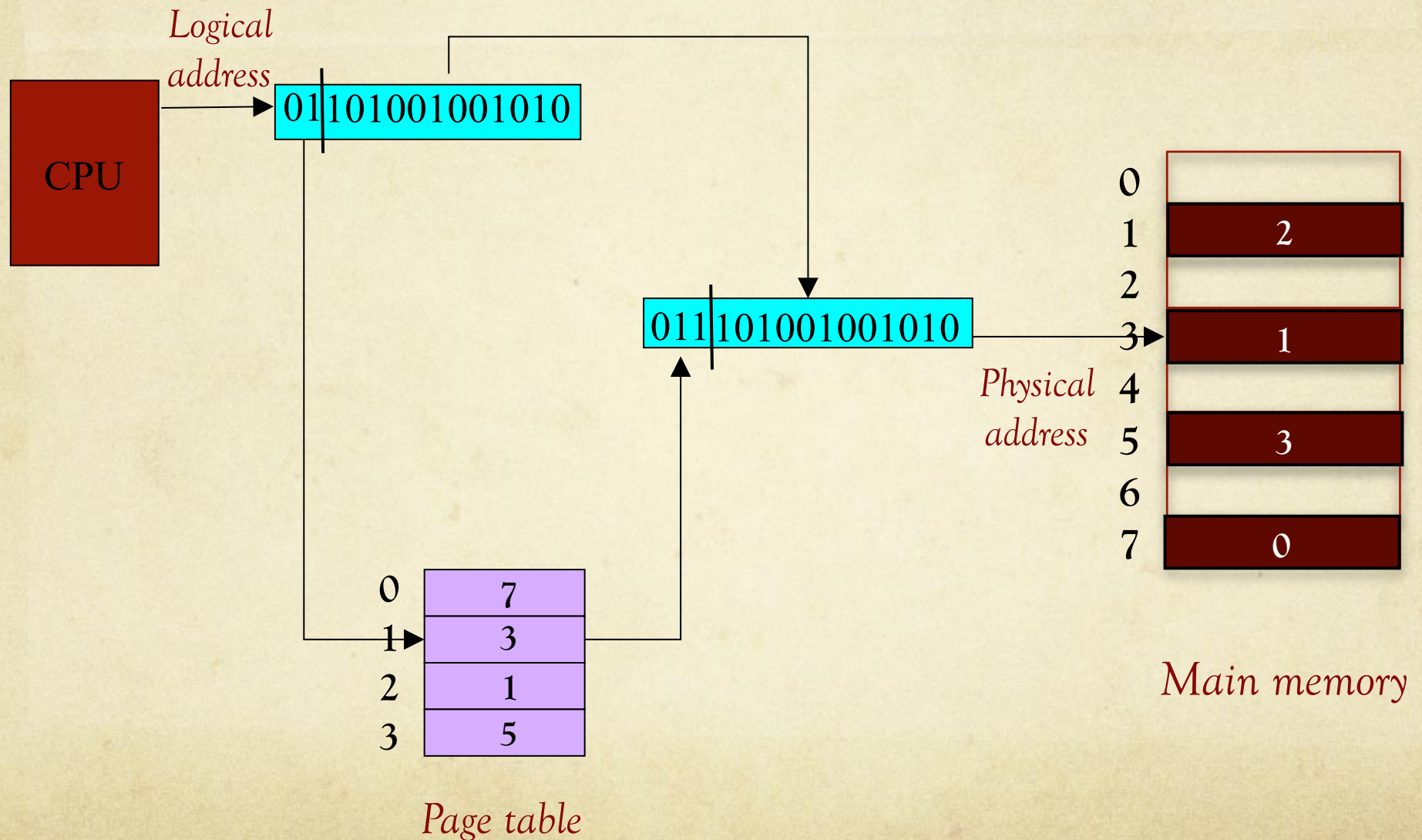
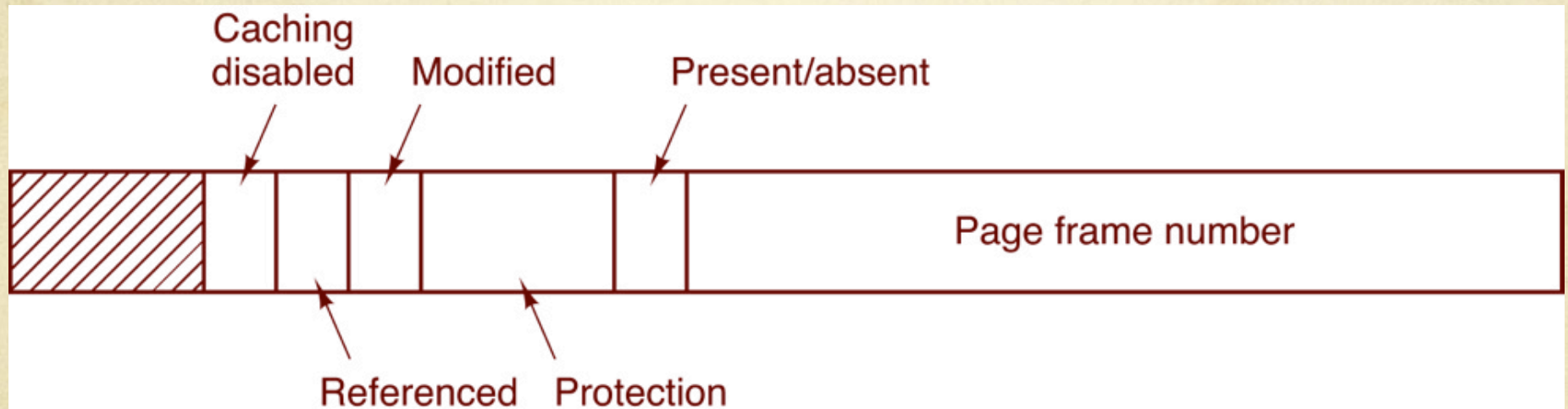


Address translation in paged systems



Page table entry structure

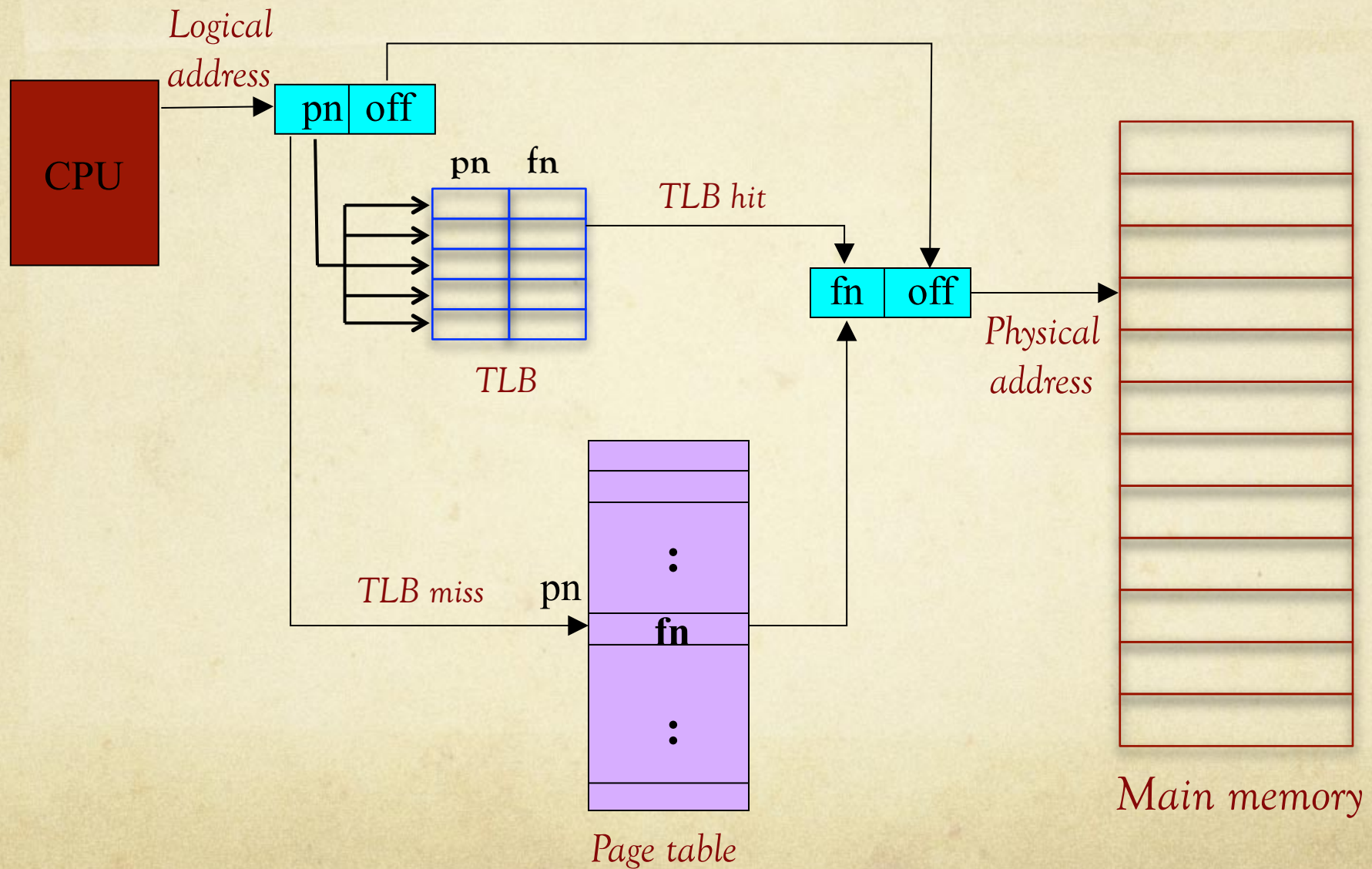
- ◆ Structure is dependent on system
- ◆ Let's look at one possible structure



More about paging

- ◆ Page table itself stored in *main memory*
 - ◆ *Expensive* to access it for each memory request
- ◆ Solution → *cache* subset of page table entries
- ◆ Translation look-aside buffer (*TLB*)
 - ◆ Small set of *associative* registers
 - ◆ Provides *fast* access

Address translation with TLB



We still have one restrictive assumption....

- ◆ *All* process pages must be in *main memory* to execute process
 - ◆ Limits *num processes* in main memory & process *data size*
- ◆ Observation
 - ◆ Process does not use *all* its data *all* the time
- ◆ Better option
 - ◆ Keep only *subset* of process pages in main memory
 - ◆ Bring others as needed
 - ◆ This is called **demand paging**

Demand paging

- ◆ When process requests for data
 - ◆ Check if page with data *already* in main memory
 - ◆ If yes, proceed as usual
 - ◆ If not, **page fault** is set to occur
- ◆ When page fault occurs
 - ◆ *Load* missing page into main memory
 - ◆ May add *TLB* entry for new page
 - ◆ *Restart* instruction that caused page fault

A consequence of demand paging...

- ◆ ...process data *not limited* to physical main memory size!
 - ◆ I.e., size of process virtually unlimited
- ◆ This is referred to as **virtual memory**
 - ◆ Did not support this in our earlier *examples*!
- ◆ What can we do?
 - ◆ Allow more logical *pages* in process than physical *page frames*
 - ◆ Logical page # can be longer physical page frame #
 - ◆ Logical (*virtual*) address can be *longer* than physical address

Memory System

- ◆ 32KB (2^{15}) main memory
 - ◆ 15 bit physical address
- ◆ 64KB (2^{16}) logical address space
 - ◆ 16 bit logical address
- ◆ 4KB (2^{12}) page frame /page
 - ◆ 12 bit offset [*LSB*]
 - ◆ $2^{15} / 2^{12} = 2^3$ page frames
 - ◆ 3 bit page frame number
 - ◆ $2^{16} / 2^{12} = 2^4$ pages
 - ◆ 4 bit logical page number